

HTML II — Formularios, tablas, accesibilidad práctica y elementos interactivos útiles

En esta sesión

En esta segunda sesión de HTML vas a trabajar una parte especialmente importante del desarrollo de interfaces reales: la captura y la presentación de datos.

Aquí vas a estudiar cómo construir formularios útiles, cómo representar información tabular de forma correcta, cómo aplicar accesibilidad práctica desde el propio HTML y cómo aprovechar algunos elementos nativos que resuelven necesidades reales sin recurrir desde el principio a soluciones más complejas.

1. Introducción

En la sesión anterior trabajaste la estructura general del documento, la semántica, la jerarquía del contenido, los enlaces, las listas y las imágenes. Esa base era imprescindible porque toda interfaz necesita una estructura clara.

En esta sesión el foco se desplaza hacia situaciones muy habituales en cualquier producto digital:

- pedir datos a la persona usuaria;
- mostrar información estructurada;
- construir pantallas que permitan interactuar con un sistema;
- crear bloques desplegables, indicadores de progreso o estructuras reutilizables;
- pensar en accesibilidad no como teoría abstracta, sino como parte directa del HTML.

Idea clave

Si la sesión anterior respondía a la pregunta “¿cómo organizo el contenido de una página?”, esta sesión responde a otra muy importante: “¿cómo hago que esa página sirva realmente para interactuar con datos y con personas usuarias reales?”.

1.1. Por qué esta sesión es clave en desarrollo web real

En casi cualquier proyecto web aparecen una o varias de estas piezas:

Elemento	Ejemplos reales
Formularios	login, registro, contacto, filtros, búsquedas, alta de recursos, edición de perfil
Tablas	listados de usuarios, pedidos, reservas, métricas, inventario
Elementos interactivos nativos	FAQs, desplegados, barras de progreso, diálogos
Accesibilidad práctica	labels correctos, ayudas, encabezados, orden lógico, atributos ARIA puntuales

Aprender bien estos temas te ahorra muchos problemas posteriores en CSS, JavaScript y frameworks.

2. Formularios: propósito y estructura básica

Los formularios permiten **capturar, organizar y enviar información**. Son una de las piezas más habituales de cualquier interfaz.

2.1. ¿Qué es un formulario?

Un formulario es una estructura HTML que agrupa controles de entrada para recoger datos. Puede utilizarse para muchas tareas:

- iniciar sesión;
- registrarse;
- enviar un mensaje;
- buscar contenido;
- aplicar filtros;
- crear o editar registros;
- completar un proceso de compra o reserva.

2.2. El elemento `<form>`

El contenedor principal de un formulario es `form`.

```
<form>
  <label for="name">Nombre</label>
  <input id="name" name="name" type="text">
</form>
```

2.3. Atributos importantes de form

Atributo	Función
action	Indica a dónde se enviarán los datos
method	Indica cómo se enviarán los datos
autocomplete	Sugiere si el navegador debe ayudar a completar campos

Ejemplo básico

```
<form action="/register" method="post">
  <label for="email">Correo electrónico</label>
  <input id="email" name="email" type="email">
  <button type="submit">Crear cuenta</button>
</form>
```

2.4. method : get y post

Los dos valores más habituales son:

Método	Uso típico
get	Búsquedas, filtros, operaciones de lectura
post	Envío de datos que crean o modifican información

Idea práctica

En una interfaz real, un formulario de búsqueda suele usar `get`, mientras que un formulario de alta o registro suele usar `post`.

2.5. Qué debes entender en esta fase

En esta sesión no necesitas profundizar aún en todo el flujo de servidor. Lo importante aquí es comprender:

- cómo se estructura un formulario;
- cómo se relacionan sus campos;
- qué papel tiene cada atributo;
- y por qué un formulario bien construido es más usable y más accesible.

Referencias del apartado 2

- MDN Web Docs. *Your first form*.
 - MDN Web Docs. *The form element*.
 - MDN Web Docs. *Sending form data*.
-

3. Controles principales de formulario

3.1. `<label>`

El elemento `label` identifica un campo. Es uno de los elementos más importantes de un formulario bien construido.

```
<label for="username">Nombre de usuario</label>
<input id="username" name="username" type="text">
```

El atributo `for` del `label` debe coincidir con el `id` del control correspondiente.

Error frecuente

Un `placeholder` no sustituye a un `label`. El campo debe poder entenderse aunque el texto de ayuda desaparezca al escribir.

3.2. `<input>`

`input` es uno de los controles más versátiles de HTML. Su comportamiento cambia según el valor de `type`.

```
<input id="email" name="email" type="email">
```

3.3. `<textarea>`

Sirve para introducir texto más largo.

```
<label for="message">Mensaje</label>
<textarea id="message" name="message"></textarea>
```

3.4. <select> y <option>

Permiten elegir una opción entre varias predefinidas.

```
<label for="level">Nivel</label>
<select id="level" name="level">
  <option value="beginner">Principiante</option>
  <option value="intermediate">Intermedio</option>
  <option value="advanced">Avanzado</option>
</select>
```

3.5. <button>

Representa un botón interactivo. En formularios es importante indicar el tipo cuando convenga.

Tipo	Función
submit	Envía el formulario
reset	Restablece los valores
button	Botón genérico sin envío automático

```
<button type="submit">Enviar</button>
```

3.6. Ejemplo base de formulario

```
<form action="/contact" method="post">
  <label for="full-name">Nombre completo</label>
  <input id="full-name" name="fullName" type="text">

  <label for="message">Mensaje</label>
  <textarea id="message" name="message"></textarea>

  <button type="submit">Enviar mensaje</button>
</form>
```

Referencias del apartado 3

- MDN Web Docs. *The label element*.
 - MDN Web Docs. *The input element*.
 - MDN Web Docs. *The textarea element*.
 - MDN Web Docs. *The select element*.
 - MDN Web Docs. *The button element*.
-

4. Tipos de `input` más usados en la práctica

El elemento `input` cambia mucho según el valor del atributo `type`. Elegir bien el tipo mejora usabilidad, validación nativa y adaptación a dispositivos.

4.1. Tipos más habituales

<code>type</code>	Uso real más frecuente
<code>text</code>	Campos generales de texto
<code>email</code>	Correo electrónico
<code>password</code>	Contraseñas
<code>number</code>	Números
<code>checkbox</code>	Activación/desactivación de una opción
<code>radio</code>	Elección de una sola opción dentro de un grupo
<code>file</code>	Subida de archivos
<code>date</code>	Selección de fecha
<code>search</code>	Buscadores
<code>url</code>	Direcciones web
<code>tel</code>	Números de teléfono

4.2. Ejemplos básicos

`text`

```
<input id="first-name" name="firstName" type="text">
```

email

```
<input id="email" name="email" type="email">
```

password

```
<input id="password" name="password" type="password">
```

number

```
<input id="age" name="age" type="number">
```

checkbox

```
<input id="terms" name="terms" type="checkbox">  
<label for="terms">Acepto las condiciones</label>
```

radio

```
<input id="online" name="mode" type="radio" value="online">  
<label for="online">Online</label>  
  
<input id="onsite" name="mode" type="radio" value="onsite">  
<label for="onsite">Presencial</label>
```

file

```
<input id="cv" name="cv" type="file">
```

date

```
<input id="start-date" name="startDate" type="date">
```

search

```
<input id="search" name="search" type="search">
```

url

```
<input id="portfolio" name="portfolio" type="url">
```

tel

```
<input id="phone" name="phone" type="tel">
```

4.3. Elegir bien el tipo importa

Elegir el tipo correcto aporta ventajas reales:

- validación nativa;
- teclado más adecuado en móvil;
- mejor comprensión del dato esperado;
- mejor experiencia general.

Regla práctica

No uses `text` para todo. Si el dato tiene un tipo específico, usa el tipo específico.

Referencias del apartado 4

- MDN Web Docs. *HTML input types*.
- MDN Web Docs. *The input element*.

5. Relación entre `label`, `for`, `id` y `name`

Esta relación es una de las claves de cualquier formulario bien hecho.

5.1. `label` y `id`

El `label` debe señalar con claridad a qué campo describe.

```
<label for="user-email">Correo electrónico</label>
<input id="user-email" name="email" type="email">
```

Cuando haces clic sobre el `label`, el foco se dirige al campo asociado. Esto mejora:

- usabilidad;
- accesibilidad;
- claridad visual y estructural.

5.2. El atributo `name`

`name` indica el nombre con el que el valor del campo se enviará en el formulario.

```
<input id="user-email" name="email" type="email">
```

Idea técnica

`id` sirve para identificar el elemento dentro del documento; `name` sirve para identificar el dato dentro del formulario.

5.3. `placeholder` y sus límites

`placeholder` puede sugerir un ejemplo o una pista breve.

```
<input id="city" name="city" type="text" placeholder="Ejemplo: Madrid">
```

Pero no debe utilizarse como sustituto de una etiqueta visible.

5.4. Ejemplo correcto

```
<label for="company">Empresa</label>
<input
  id="company"
  name="company"
  type="text"
  placeholder="Ejemplo: TechLab"
>
```

Referencias del apartado 5

- MDN Web Docs. *The label element*.
 - MDN Web Docs. *HTML attribute: name*.
 - MDN Web Docs. *HTML attribute: placeholder*.
-

6. Agrupación semántica y ayuda al usuario

6.1. fieldset

Agrupar campos relacionados dentro de un formulario.

```
<fieldset>
  <legend>Preferencias de modalidad</legend>

  <input id="mode-online" name="mode" type="radio" value="online">
  <label for="mode-online">Online</label>

  <input id="mode-onsite" name="mode" type="radio" value="onsite">
  <label for="mode-onsite">Presencial</label>
</fieldset>
```

6.2. legend

Da nombre al grupo definido por `fieldset`. Es especialmente útil cuando agrupas radios, checkboxes o campos temáticamente relacionados.

6.3. datalist

`datalist` permite sugerir opciones a un campo de entrada.

```
<label for="browser">Navegador</label>
<input id="browser" name="browser" list="browser-options">

<datalist id="browser-options">
  <option value="Chrome">
  <option value="Firefox">
  <option value="Safari">
</datalist>
```

Criterio de uso

`datalist` puede ser útil como complemento, pero no siempre conviene apoyarse en él para flujos críticos. En esta sesión basta con conocerlo y saber qué resuelve.

6.4. output

`output` representa el resultado de una operación o cálculo.

```
<form>
  <label for="quantity">Cantidad</label>
  <input id="quantity" name="quantity" type="number" value="1">

  <output for="quantity">1</output>
</form>
```

En esta sesión no necesitas explotarlo a fondo; basta con comprender que existe y qué función puede tener.

Referencias del apartado 6

- MDN Web Docs. *The fieldset element*.
- MDN Web Docs. *The legend element*.
- MDN Web Docs. *The datalist element*.
- MDN Web Docs. *The output element*.

7. Validación nativa de HTML

HTML incorpora mecanismos de validación muy útiles sin necesidad de JavaScript.

7.1. Atributos de validación más comunes

Atributo	Función
required	Hace obligatorio un campo
minlength	Establece una longitud mínima
maxlength	Establece una longitud máxima
min	Valor mínimo permitido
max	Valor máximo permitido
pattern	Expresión de formato esperada
readonly	Campo visible pero no editable
disabled	Campo desactivado

7.2. Ejemplos

Campo obligatorio

```
<input id="email" name="email" type="email" required>
```

Longitud mínima y máxima

```
<input  
  id="username"  
  name="username"  
  type="text"  
  minlength="4"  
  maxlength="20"  
>
```

Rango numérico

```
<input id="age" name="age" type="number" min="18" max="99">
```

Patrón

```
<input  
  id="postal-code"  
  name="postalCode"  
  type="text"  
  pattern="[0-9]{5}"  
>
```

Solo lectura

```
<input id="country" name="country" type="text" value="España" readonly>
```

Desactivado

```
<input id="coupon" name="coupon" type="text" disabled>
```

7.3. Ventajas de la validación nativa

- reduce errores simples;
- guía a la persona usuaria;
- refuerza la semántica del campo;

- mejora la experiencia básica sin código extra.

7.4. Límites de la validación nativa

No sustituye validaciones más complejas ni comprobaciones del lado del servidor. Aun así, es una base excelente y conviene usarla.

Enfoque correcto

La validación nativa no es “poca cosa”. Es una primera capa muy útil y profesional.

Referencias del apartado 7

- MDN Web Docs. *Client-side form validation*.
- MDN Web Docs. *Constraint validation*.

8. Accesibilidad práctica en formularios

Aquí la accesibilidad deja de ser una idea abstracta y pasa a afectar directamente a cómo escribes el HTML.

8.1. `label` no es opcional

Cada campo debe poder entenderse claramente. El `label` es la forma principal y correcta de hacerlo.

8.2. Ayudas e instrucciones

A veces un campo necesita una explicación adicional.

```
<label for="password">Contraseña</label>
<p id="password-help">Debe tener al menos 8 caracteres.</p>
<input
  id="password"
  name="password"
  type="password"
  aria-describedby="password-help"
>
```

8.3. `aria-describedby`

Relaciona un campo con una ayuda, instrucción o descripción adicional.

8.4. `aria-label`

Debe usarse cuando no existe una etiqueta visible adecuada y necesitas proporcionar una etiqueta accesible.

```
<input type="search" name="search" aria-label="Buscar cursos">
```

Criterio importante

Primero intenta resolver el problema con HTML semántico correcto y labels visibles. Recurre a ARIA cuando realmente haga falta.

8.5. Orden lógico y claridad

Un formulario debe tener sentido en su orden natural de lectura y foco. No debe depender del diseño visual para entenderse.

8.6. Errores comprensibles

Aunque aquí no entres todavía en toda la lógica de errores dinámicos, debes construir formularios que ya sean claros desde la estructura.

Referencias del apartado 8

- W3C WAI. *Forms tutorial*.
- MDN Web Docs. *ARIA: aria-describedby*.
- MDN Web Docs. *ARIA: aria-label*.

9. Tablas de datos bien construidas

Las tablas siguen siendo muy útiles cuando necesitas mostrar **datos tabulares reales**.

9.1. Cuándo usar una tabla

Usa una tabla cuando la información esté organizada naturalmente en filas y columnas.

Ejemplos:

- listado de alumnos;

- reservas;
- agenda;
- inventario;
- métricas;
- horarios.

9.2. Cuándo no usar una tabla

No uses tablas para maquetar páginas completas ni para construir layouts generales. Ese enfoque pertenece a prácticas antiguas que no preparan para desarrollo actual.

9.3. Estructura básica de una tabla

Elemento	Función
<code>table</code>	Contenedor de la tabla
<code>caption</code>	Título o descripción breve
<code>thead</code>	Cabecera
<code>tbody</code>	Cuerpo principal
<code>tfoot</code>	Pie de la tabla
<code>tr</code>	Fila
<code>th</code>	Celda de encabezado
<code>td</code>	Celda de datos

9.4. Ejemplo base

```
<table>
  <caption>Próximos talleres disponibles</caption>
  <thead>
    <tr>
      <th scope="col">Taller</th>
      <th scope="col">Fecha</th>
      <th scope="col">Plazas</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Introducción a HTML</td>
```

```

        <td>12/04/2026</td>
        <td>20</td>
    </tr>
    <tr>
        <td>CSS Layouts</td>
        <td>19/04/2026</td>
        <td>15</td>
    </tr>
</tbody>
</table>

```

9.5. caption

Describe de forma breve el contenido de la tabla. Es útil para contexto y accesibilidad.

9.6. scope

Ayuda a relacionar encabezados y datos. Sus valores más habituales son:

Valor	Función
col	Encabezado de columna
row	Encabezado de fila

9.7. Ejemplo con encabezado de fila

```

<table>
  <caption>Estado de servidores</caption>
  <thead>
    <tr>
      <th scope="col">Servidor</th>
      <th scope="col">Estado</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">API principal</th>
      <td>Operativo</td>
    </tr>
  </tbody>
</table>

```


Referencias del apartado 9

- MDN Web Docs. *HTML tables*.
 - MDN Web Docs. *The table element*.
 - W3C WAI. *Tables tutorial*.
-

10. Accesibilidad en tablas

10.1. Encabezados claros

Las tablas deben definir correctamente qué celdas funcionan como encabezados.

10.2. `caption` aporta contexto

Una tabla sin contexto es más difícil de interpretar. `caption` ayuda a entender qué representa.

10.3. Evitar usos impropios

Una tabla de maquetación crea confusión estructural y problemas de accesibilidad. Si el contenido no es tabular, la tabla no es la herramienta adecuada.

Regla práctica

Una tabla debe usarse para datos que de verdad necesitan leerse en relación fila-columna.

Referencias del apartado 10

- W3C WAI. *Tables concepts*.
 - MDN Web Docs. *Accessible table structures*.
-

11. Elementos interactivos nativos útiles

HTML incluye elementos que resuelven necesidades reales sin requerir desde el principio componentes complejos.

11.1. `details` y `summary`

Permiten crear bloques desplegados de forma nativa.

```
<details>
  <summary>¿Necesito experiencia previa?</summary>
  <p>No. Este taller empieza desde una base introductoria.</p>
</details>
```

Casos de uso reales

- preguntas frecuentes;
- bloques de ayuda;
- paneles simples de información;
- aclaraciones adicionales.

Valor práctico

Para FAQs sencillas o contenido auxiliar, `details` y `summary` pueden resolver mucho con muy poco.

11.2. progress

Representa el progreso de una tarea.

```
<label for="course-progress">Progreso del curso</label>
<progress id="course-progress" value="60" max="100">60%</progress>
```

11.3. meter

Representa un valor dentro de un rango conocido, no necesariamente un progreso temporal.

```
<label for="disk-usage">Uso del almacenamiento</label>
<meter id="disk-usage" min="0" max="100" value="72">72%</meter>
```

11.4. Diferencia entre progress y meter

Elemento	Uso principal
<code>progress</code>	Avance de una tarea o proceso
<code>meter</code>	Medida dentro de un rango conocido

11.5. dialog

Representa un cuadro de diálogo nativo.

```
<dialog open>
  <p>Tu sesión expirará pronto.</p>
  <button>Entendido</button>
</dialog>
```

En esta sesión basta con entender:

- que existe;
- que puede ser útil para ciertos modales o avisos;
- que su uso debe hacerse con criterio y pruebas adecuadas.

Alcance de esta sesión

No necesitas profundizar todavía en toda la lógica de apertura, cierre y control avanzado de `dialog`. Aquí basta con conocer su papel y su potencial.

Referencias del apartado 11

- MDN Web Docs. *The details element*.
- MDN Web Docs. *The summary element*.
- MDN Web Docs. *The progress element*.
- MDN Web Docs. *The meter element*.
- MDN Web Docs. *The dialog element*.

12. HTML incrustado e integración externa

12.1. `iframe`

Permite incrustar contenido externo dentro de una página.

```
<iframe
  src="https://example.com"
  title="Mapa de localización de la academia"
></iframe>
```

Usos frecuentes

- mapas;
- vídeos externos;
- recursos incrustados;
- ciertos paneles o widgets.

Criterio de uso

`iframe` es útil en contextos concretos, pero no debe convertirse en una solución automática para cualquier integración.

12.2. `script`

Permite enlazar o incluir JavaScript.

```
<script src="./app.js"></script>
```

12.3. `link`

Permite enlazar recursos externos, especialmente hojas de estilo o iconos.

```
<link rel="stylesheet" href="./styles.css">  
<link rel="icon" href="./favicon.ico">
```

12.4. Idea clave

Aunque esta sesión sigue centrada en HTML, debes ir entendiendo cómo el documento se conecta con otras capas del proyecto.

Referencias del apartado 12

- MDN Web Docs. *The iframe element*.
- MDN Web Docs. *The script element*.
- MDN Web Docs. *The link element*.

13. Accesibilidad ampliada en HTML

13.1. `aria-hidden`

Indica que un elemento debe quedar oculto para ciertas tecnologías de asistencia.

```
<span aria-hidden="true">★</span>
```

Se usa con cuidado, normalmente en elementos decorativos o redundantes.

13.2. `role`

Puede indicar el papel de un elemento cuando la semántica nativa no es suficiente.

```
<div role="status">Guardado correctamente</div>
```

13.3. Principio básico de uso de ARIA

Regla esencial

Primero usa HTML correcto. Después, si hace falta, complementa con ARIA. No utilices ARIA para “arreglar” una estructura mal planteada.

Referencias del apartado 13

- W3C WAI. *ARIA basics*.
- MDN Web Docs. *ARIA: aria-hidden*.
- MDN Web Docs. *ARIA roles*.

14. Buenas prácticas actuales en formularios y tablas

14.1. En formularios

- usa `label` visibles;
- elige bien el tipo de `input` ;
- usa validación nativa cuando tenga sentido;
- no sustituyas etiquetas por `placeholder` ;
- agrupa campos relacionados con `fieldset` y `legend` cuando proceda;
- piensa en claridad antes que en cantidad de campos.

14.2. En tablas

- usa tablas solo para datos tabulares reales;
- define encabezados correctamente;

- añade `caption` cuando ayude a contextualizar;
- evita tablas enormes y confusas si otra estructura sería mejor.

14.3. En elementos interactivos

- usa elementos nativos cuando resuelvan bien el problema;
- no compliques con componentes más pesados si una solución HTML sencilla basta;
- revisa siempre la claridad estructural y la accesibilidad.

Tabla de errores frecuentes

Error	Problema	Enfoque correcto
Usar <code>placeholder</code> como única etiqueta	El campo pierde claridad	Mantener <code>label</code> visible
Usar <code>text</code> para todo	Peor experiencia y validación	Elegir el tipo de <code>input</code> adecuado
Crear formularios sin <code>name</code>	Los datos no se identifican bien	Añadir <code>name</code> coherente
Usar tablas para maquetar	Mala estructura y peor accesibilidad	Reservar tablas para datos reales
Ignorar <code>scope</code> y <code>caption</code>	Tabla menos comprensible	Definir encabezados y contexto
Abusar de ARIA	Complica sin necesidad	Priorizar HTML semántico correcto

Referencias del apartado 14

- MDN Web Docs. *Forms best practices*.
- W3C WAI. *Forms tutorial*.
- W3C WAI. *Tables tutorial*.

15. Análisis técnico avanzado

15.1. Formularios bien hechos reducen deuda técnica

Un formulario mal planteado genera problemas en cadena:

- CSS más difícil de aplicar;
- validaciones más torpes;
- errores de usabilidad;
- peor accesibilidad;
- lógica JavaScript más frágil.

15.2. Elegir bien el HTML simplifica el resto del sistema

A menudo la diferencia entre una interfaz robusta y una interfaz incómoda empieza en decisiones aparentemente pequeñas:

- elegir bien el tipo de `input` ;
- escribir `label` claros;
- agrupar semánticamente;
- usar tablas solo cuando corresponde;
- no inventar con `div` lo que HTML ya resuelve.

15.3. Lo nativo suele ser una base excelente

En desarrollo moderno existe una tendencia a reemplazar demasiado pronto soluciones HTML nativas por implementaciones personalizadas. Eso no siempre es una mejora.

Elementos como:

- `details` ;
- `summary` ;
- `progress` ;
- `meter` ;
- `dialog` ;

demuestran que HTML sigue aportando utilidades reales más allá de “texto y estructura”.

15.4. Limitaciones de esta sesión

En esta sesión todavía no profundizas en:

- validación avanzada con JavaScript;
- formularios controlados en React;
- tablas dinámicas complejas;
- gestión de estado;
- manipulación avanzada de diálogos.

Todo eso vendrá más adelante. Aquí el objetivo es dominar una base sólida y actual de HTML útil.

16. Conclusión

En esta segunda sesión de HTML has trabajado una parte esencial del desarrollo de interfaces reales: formularios, tablas, accesibilidad práctica y algunos elementos interactivos nativos que conviene conocer desde el principio.

Has visto que HTML no se limita a describir contenido estático. También permite:

- recoger datos;
- estructurar información compleja;
- mejorar la experiencia de uso;
- y resolver necesidades reales de interacción con soluciones nativas y mantenibles.

Esta base será especialmente importante cuando empieces a maquetar con CSS y, más adelante, cuando construyas interfaces más complejas con JavaScript y React.

17. Resumen de ideas clave

Qué debes retener de esta sesión

- Los formularios son una pieza central en interfaces reales.
- `label`, `id`, `for` y `name` cumplen funciones distintas y todas son importantes.
- Debes elegir el tipo de `input` adecuado según el dato esperado.
- HTML ofrece validación nativa útil mediante atributos como `required`, `minlength`, `pattern`, `min` o `max`.
- `placeholder` no sustituye a `label`.
- `fieldset` y `legend` ayudan a agrupar campos con sentido.
- Las tablas deben usarse para datos tabulares reales, no para maquetar.
- `caption`, `th` y `scope` mejoran comprensión y accesibilidad en tablas.
- `details`, `summary`, `progress`, `meter` y `dialog` son elementos nativos con valor práctico.
- La accesibilidad en formularios y tablas empieza por un HTML correcto.
- ARIA complementa al HTML, pero no debe sustituir una buena estructura semántica.

18. Referencias documentales generales

1. MDN Web Docs. *Your first form*.
 2. MDN Web Docs. *The form element*.
 3. MDN Web Docs. *The input element*.
 4. MDN Web Docs. *Client-side form validation*.
 5. MDN Web Docs. *HTML tables*.
 6. W3C WAI. *Forms tutorial*.
 7. W3C WAI. *Tables tutorial*.
 8. MDN Web Docs. *The details element*.
 9. MDN Web Docs. *The dialog element*.
 10. MDN Web Docs. *HTML: A good basis for accessibility*.
-

19. Mini glosario de la sesión

Término	Definición breve
Formulario	Estructura HTML para capturar y enviar datos
<code>label</code>	Etiqueta descriptiva asociada a un campo
<code>name</code>	Nombre con el que se identifica el dato enviado
Validación nativa	Comprobaciones básicas incorporadas por HTML
<code>fieldset</code>	Agrupación semántica de controles relacionados
<code>legend</code>	Título del grupo definido por <code>fieldset</code>
Tabla de datos	Estructura para información organizada en filas y columnas
<code>caption</code>	Descripción breve de una tabla
<code>scope</code>	Relación entre encabezados y datos en una tabla
<code>details</code>	Elemento desplegable nativo
<code>progress</code>	Indicador de avance
<code>meter</code>	Medida dentro de un rango conocido
<code>dialog</code>	Cuadro de diálogo nativo

20. Fragmento de ejemplo integrador

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Registro de taller técnico</title>
    <meta
      name="description"
      content="Formulario de inscripción, tabla de sesiones y preguntas
frecuentes del taller técnico."
    >
    <link rel="icon" href="./assets/favicon.ico">
  </head>
  <body>
    <header>
      <h1>Taller intensivo de HTML y CSS</h1>
      <p>Inscripción abierta para la próxima edición.</p>
    </header>

    <main>
      <section aria-labelledby="registration-title">
        <h2 id="registration-title">Formulario de inscripción</h2>

        <form action="/register" method="post">
          <label for="full-name">Nombre completo</label>
          <input id="full-name" name="fullName" type="text" required>

          <label for="email">Correo electrónico</label>
          <input id="email" name="email" type="email" required>

          <fieldset>
            <legend>Modalidad</legend>

            <input id="online" name="mode" type="radio" value="online">
            <label for="online">Online</label>

            <input id="onsite" name="mode" type="radio" value="onsite">
            <label for="onsite">Presencial</label>
          </fieldset>

          <label for="message">Motivación</label>
          <textarea id="message" name="message" aria-describedby="message-
```

```
help"></textarea>
    <p id="message-help">Cuéntanos brevemente por qué quieres
participar.</p>

    <button type="submit">Enviar solicitud</button>
</form>
</section>

<section aria-labelledby="sessions-title">
    <h2 id="sessions-title">Sesiones disponibles</h2>

    <table>
        <caption>Próximas convocatorias del taller</caption>
        <thead>
            <tr>
                <th scope="col">Edición</th>
                <th scope="col">Fecha</th>
                <th scope="col">Plazas</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td>Abril</td>
                <td>12/04/2026</td>
                <td>20</td>
            </tr>
            <tr>
                <td>Mayo</td>
                <td>10/05/2026</td>
                <td>18</td>
            </tr>
        </tbody>
    </table>
</section>

<section aria-labelledby="faq-title">
    <h2 id="faq-title">Preguntas frecuentes</h2>

    <details>
        <summary>¿Necesito experiencia previa?</summary>
        <p>No. El taller está pensado para empezar desde una base
introductoria.</p>
    </details>

    <details>
        <summary>¿Cómo sabré mi progreso?</summary>
```

```
<p>Recibirás seguimiento continuo del avance del curso.</p>
<label for="progress-course">Progreso estimado</label>
<progress id="progress-course" value="45" max="100">45%</progress>
</details>
</section>
</main>

<footer>
  <p>Contacto: <a
href="mailto:hola@academiadev.com">hola@academiadev.com</a></p>
</footer>
</body>
</html>
```